High Performance Parallel Computing Assignment 6

António Maschio ; Dimitrios Anastasiou ; Georgios Sevastakis

March 2024

Task 1: Parallelization

In this project, we undertook the task of parallelizing a climate model using OpenMPI. Initially, we decomposed the system into multiple smaller 1D or 2D domains, each representing a distinct portion of the global simulation space. Surrounding each domain, we introduced a ghost layer to facilitate communication and computation across domain boundaries. Notably, for this specific simulation, corner ghost cells were deemed unnecessary, and thus, we only included a single-cell-length ghost layer for computational purposes.

Following the domain decomposition, we populated each domain with data extracted from the original system. To get the correct data for the ghost lines, firstly we populated the data with data from the boundary of the own domain, and then we exchanged the data between neighbor ghost layers. With these steps, the simulation calculations are ready to be performed on the system. In this case Radiation, Energy emission, and Diffusion. After calculation, we gather all information in rank 0 and process it.

In our implementation, we were able to implement 1D domain decomposition successfully, while the 2D failed. Unfortunately, we were also not able to bench–mark more than just the medium sized model due to the shockingly lengthy queue on ERDA. As far as the hybrid implementation is concerned, we put parallel for directives on the for–loops of the code in the 1D case, namely in the ones seen in the *radiation*, *energy_emission* and *diffuse* functions. No other clauses other than the *collapse* were deemed necessary.

Again, due to the lengthy queue on ERDA we were not able to run and benchmark our hybrid code. We would expect a good speed up for low numbers of MPI–processes, so that our for–loops would be lengthy enough (and therefore, worth parallelizing). For a greater number of MPI–processes, the overhead of the parallel omp would be having an impact on the elapsed time and we would expect a drop in performance.

Task 2: Strong scaling

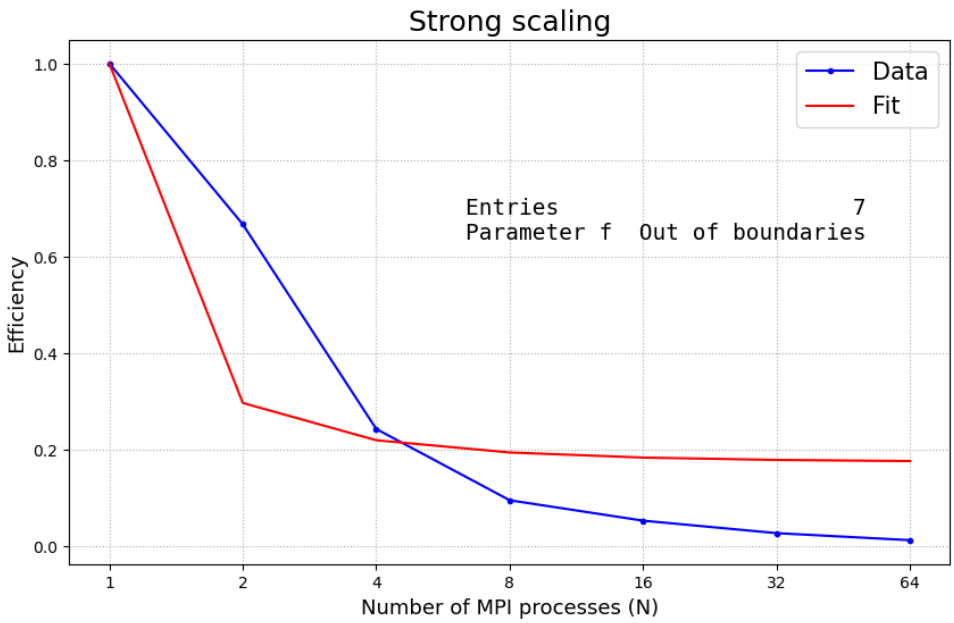

Figure 1: Speed up of the 1D slab geometry code as a function of the number of MPI–processes (here: 1 node) with Ahmdahl's law fit included for the medium sized model for 500 iterations.

Having calculated the Efficiency E by dividing the elapsed time of the sequential version (1 MPI–process) with that of the parallelized version (N MPI–processes) and by the number of MPI–processes, we plotted the graph above and fitted Ahmdahl's law in the strong scaling context.

Efficiency and its relation to Ahmdahl's law is given by the following formula

$$E = \frac{S_{Ahm.}}{N}(N) = \frac{1}{1 - f + \frac{f}{N}} \tag{1}$$

where $S_{Ahm.}$ is the Speed up, f is the parallel fraction and N is the number of MPI–processes.

By fitting Equation 1 we were able to compute the parallel fraction f, which was found to be less than zero.

From the aforementioned result and by looking at Figure 1, we conclude that our implementation is not efficient, even though we are running the code on one single node (64 cores). This means that by adding more MPI–processes, our code is performing worse. Had we used more cores, we would expect an even worse performance related to communication, since by accessing another node (NUMA domain), the communication cost is heavier.

One of the main reasons is the fact that the trade–off between the communication penalty (something that Amhdahl's law does not take into account) and the efficiency of parallelization tilts to the former side. Apart from that, another thing that makes our implementation under-perform is the fact that each rank loads the whole data in the beginning leading to a grave time penalty. Moreover, even though we did not include an output file while bench–marking, we did still gather our local data in the *temp* variable via the MPI_Reduce command. This proved to be time consuming not only due to the fact that the MPI_Reduce had to sum all of the full–sized data vectors, but also due to the two for–loops just above it, especially the first one, which had to iterate through all of the elements of the global data vector.

Appendices

A C++ code

```cpp
#include <vector>
#include <iostream>
#include <H5Cpp.h>
#include <chrono>
#include <cmath>
#include <numeric>
#include <mpi.h>

// Get the number of processes
int mpi_size;

// Get the rank of the process
int mpi_rank;
MPI_Datatype column;
/** Representation of a flat world */
class World {
public:
    // The current world time of the world
    double time;
    // The size of the world in the latitude dimension and the global size
    uint64_t latitude, global_latitude;
    // The size of the world in the longitude dimension
    uint64_t longitude, global_longitude;
    // The offset for this rank in the latitude dimension
    long int offset_latitude;
    // The offset for this rank in the longitude dimension
    long int offset_longitude;
    // The temperature of each coordinate of the world.
    // NB: it is up to the calculation to interpret this vector in two dimension.
    std::vector<double> data;
    // The measure of the diffuse reflection of solar radiation at each world coordinate.
    // See: <https://en.wikipedia.org/wiki/Albedo>
    // NB: this vector has the same length as ‘data‘ and must be interpreted in two dimension as well.
    std::vector<double> albedo_data;

    /** Create a new flat world.
     *
     * @param latitude     The size of the world in the latitude dimension.
     * @param longitude    The size of the world in the longitude dimension.
     * @param temperature  The initial temperature (the whole world starts with the same temperature).
     * @param albedo_data  The measure of the diffuse reflection of solar radiation at each world coordinate.
     *                     This vector must have the size: ‘latitude * longitude‘.
     */
    World(uint64_t latitude, uint64_t longitude, double temperature,
          std::vector<double> albedo_data) : latitude(latitude), longitude(longitude),
                                             data(latitude * longitude, temperature),
                                             albedo_data(std::move(albedo_data)) {}
};

double checksum(World &world) {
    //
    double cs=0;
```

```
    // TODO: make sure checksum is computed globally
    // only loop *inside* data region -- not in ghostzones!
    for (uint64_t i = 1; i < world.latitude - 1; ++i)
    for (uint64_t j = 1; j < world.longitude - 1; ++j) {
        cs += world.data[i*world.longitude + j];
    }
    return cs;
}

void stat(World &world) {
    // TODO: make sure stats are computed globally
    double mint = 1e99;
    double maxt = 0;
    double meant = 0;
    double global_mint = 0;
    double global_maxt = 0;
    double global_meant = 0;
    double size = world.global_latitude * world.global_longitude;

    if(world.data.size() < world.global_latitude * world.global_longitude){
    for (uint64_t i = 1; i < world.latitude - 1; ++i)
    for (uint64_t j = 1; j < world.longitude - 1; ++j) {
        mint = std::min(mint,world.data[i*world.longitude + j]);
        maxt = std::max(maxt,world.data[i*world.longitude + j]);
        meant += world.data[i*world.longitude + j];
    }
    meant = meant / size;

    MPI_Reduce(&meant, &global_meant, 1, MPI_DOUBLE, MPI_SUM, 0, MPI_COMM_WORLD);
    MPI_Reduce(&mint, &global_mint, 1, MPI_DOUBLE, MPI_MIN, 0, MPI_COMM_WORLD);
    MPI_Reduce(&maxt, &global_maxt, 1, MPI_DOUBLE, MPI_MAX, 0, MPI_COMM_WORLD);

    }
    else{
        size = world.latitude*world.longitude;
        for (uint64_t i = 0; i < world.latitude; ++i)
        for (uint64_t j = 0; j < world.longitude; ++j) {
        mint = std::min(mint,world.data[i*world.longitude + j]);
        maxt = std::max(maxt,world.data[i*world.longitude + j]);
        meant += world.data[i*world.longitude + j];
        }
        // std::cout<<"mean is"<<meant<<"\n size is"<<size<<std::endl;
        meant = meant / size;

        global_mint = mint;
        global_maxt = maxt;
        global_meant = meant;
        }
    if(mpi_rank == 0){
    std::cout <<   "min: " << global_mint
              << ", max: " << global_maxt
              << ", avg: " << global_meant << std::endl;
    }
}

/** Exchange the ghost cells i.e. copy the second data row and column to the very
    last data row and column and vice versa.
 *
 * @param world  The world to fix the boundaries for.
 */
void exchange_ghost_cells(World &world) {
    // TODO: figure out exchange of ghost cells bewteen ranks
```

```
    // Comment: Exchange of the ghost lines by sending and receiving data.

    MPI_Request req[4];
    int tag = 0;

    int left = (mpi_rank - 1 + mpi_size) % mpi_size;
    int right = (mpi_rank +  1) % mpi_size;

    MPI_Isend(&world.data[1], 1, column, left, tag, MPI_COMM_WORLD, &req[0]);
    MPI_Isend(&world.data[world.longitude - 2], 1, column, right, tag, MPI_COMM_WORLD, &req[1]);
    MPI_Irecv(&world.data[0], 1, column, left, tag, MPI_COMM_WORLD, &req[2]);
    MPI_Irecv(&world.data[world.longitude - 1], 1, column, right, tag, MPI_COMM_WORLD, &req[3]);
    MPI_Waitall(4, &req[0], MPI_STATUSES_IGNORE);

    for (uint64_t j = 0; j < world.longitude; ++j) {
        world.data[0*world.longitude + j] = world.data[(world.latitude-2)*world.longitude + j];
        world.data[(world.latitude-1)*world.longitude + j] = world.data[1*world.longitude + j];
    }

}

/** Warm the world based on the position of the sun.
 *
 * @param world      The world to warm.
 */
void radiation(World& world) {
    double sun_angle = std::cos(world.time);
    double sun_intensity = 865.0;
    double sun_long = (std::sin(sun_angle) * (world.global_longitude / 2))
                      + world.global_longitude / 2.;
    double sun_lat = world.global_latitude / 2.;
    double sun_height = 100. + std::cos(sun_angle) * 100.;
    double sun_height_squared = sun_height * sun_height;

    for (uint64_t i = 1; i < world.latitude-1; ++i) {
        for (uint64_t j = 1; j < world.longitude-1; ++j) {
            // Euclidean distance between the sun and each earth coordinate
            double delta_lat  = sun_lat  - (i + world.offset_latitude);
            double delta_long = sun_long - (j + world.offset_longitude);
            double dist = sqrt(delta_lat*delta_lat +
                               delta_long*delta_long +
                               sun_height_squared);
            world.data[i * world.longitude + j] += \
                (sun_intensity / dist) * (1. - world.albedo_data[i * world.longitude + j]);
        }
    }
    exchange_ghost_cells(world);
}

/** Heat radiated to space
 *
 * @param world  The world to update.
 */
void energy_emmision(World& world) {
    for (uint64_t i = 0; i < world.latitude * world.longitude; ++i) {
        world.data[i] *= 0.99;
    }
```

```
}

/** Heat diffusion
 *
 * @param world  The world to update.
 */
void diffuse(World& world) {
    std::vector<double> tmp = world.data;
    for (uint64_t k = 0; k < 10; ++k) {
        for (uint64_t i = 1; i < world.latitude - 1; ++i) {
            for (uint64_t j = 1; j < world.longitude - 1; ++j) {
                // 5 point stencil
                double center = world.data[i * world.longitude + j];
                double left = world.data[(i - 1) * world.longitude + j];
                double right = world.data[(i + 1) * world.longitude + j];
                double up = world.data[i * world.longitude + (j - 1)];
                double down = world.data[i * world.longitude + (j + 1)];
                tmp[i * world.longitude + j] = (center + left + right + up + down) / 5.;
            }
        }
        std::swap(world.data, tmp);  // swap pointers for the two arrays
        exchange_ghost_cells(world); // update ghost zones
    }
}

/** One integration step at `world_time`
 *
 * @param world      The world to update.
 */
void integrate(World& world) {
    radiation(world);
    energy_emmision(world);
    diffuse(world);
}

/** Read a world model from a HDF5 file
 *
 * @param filename The path to the HDF5 file.
 * @return         A new world based on the HDF5 file.
 */
World read_world_model(const std::string& filename) {
    H5::H5File file(filename, H5F_ACC_RDONLY);
    H5::DataSet dataset = file.openDataSet("world");
    H5::DataSpace dataspace = dataset.getSpace();

    if (dataspace.getSimpleExtentNdims() != 2) {
        throw std::invalid_argument("Error while reading the model: the number of dimension must be two.");
    }

    if (dataset.getTypeClass() != H5T_FLOAT or dataset.getFloatType().getSize() != 8) {
        throw std::invalid_argument("Error while reading the model: wrong data type, must be double.");
    }

    hsize_t dims[2];
    dataspace.getSimpleExtentDims(dims, NULL);
    std::vector<double> data_out(dims[0] * dims[1]);
    dataset.read(data_out.data(), H5::PredType::NATIVE_DOUBLE, dataspace, dataspace);
```

```
    std::cout << "World model loaded -- latitude: " << (unsigned long) (dims[0]) << " , longitude: "
              << (unsigned long) (dims[1]) << std::endl;
    return World(static_cast<uint64_t>(dims[0]), static_cast<uint64_t>(dims[1]), 293.15, std::move(data_out));
}

/** Write data to a hdf5 file
 *
 *  @param group  The hdf5 group to write in
 * @param name   The name of the data
 * @param shape  The shape of the data
 * @param data   The data
 */
void write_hdf5_data(H5::Group& group, const std::string& name,
                     const std::vector <hsize_t>& shape, const std::vector<double>& data) {
    H5::DataSpace dataspace(static_cast<int>(shape.size()), &shape[0]);
    H5::DataSet dataset = group.createDataSet(name.c_str(), H5::PredType::NATIVE_DOUBLE, dataspace);
    dataset.write(&data[0], H5::PredType::NATIVE_DOUBLE);
}

/** Write a history of the world temperatures to a HDF5 file
 *s
 * @param world     world to write
 * @param filename  The output filename of the HDF5 file
 */
void write_hdf5(const World& world, const std::string& filename, uint64_t iteration) {

    static H5::H5File file(filename, H5F_ACC_TRUNC);

    H5::Group group(file.createGroup("/" + std::to_string(iteration)));
    write_hdf5_data(group, "world", {world.latitude, world.longitude}, world.data);
}

/** Simulation of a flat word climate
 *
 * @param num_of_iterations  Number of time steps to simulate
 * @param model_filename     The filename of the world model to use (HDF5 file)
 * @param output_filename    The filename of the written world history (HDF5 file)
 */
void simulate(uint64_t num_of_iterations, const std::string& model_filename, const std::string& output_filename) {

    // for simplicity, read in full model
    World global_world = read_world_model(model_filename);
    /*
    // TODO: compute offsets according to rank and domain decomposition
    // figure out size of domain for this rank
    */

    int rem, divi;

    rem = global_world.longitude % mpi_size;
    divi = global_world.longitude / mpi_size;
    // Comment: We divide our grid in 'divi' parts. If there is a remainder, we distribute it equally amongst the ranks in order.
    const uint64_t longitude = (mpi_rank < rem) ? divi + 3 : divi + 2; // one ghost cell on each end
     // Comment: We compute the offset of each rank by considering the remainder.
```

```
    const long int offset_longitude = -1 + mpi_rank * divi + ((mpi_rank < rem) ? mpi_rank : rem); // -1 because first cell is a ghostcell;

     // Comment: Latitute and longitude offset do not change.
    const uint64_t latitude  = global_world.latitude + 2;
    const long int offset_latitude  = -1;


    // copy over albedo data to local world data
    std::vector<double> albedo(longitude*latitude);
    for (uint64_t i = 1; i < latitude-1; ++i)
    for (uint64_t j = 1; j < longitude-1; ++j) {
        uint64_t k_global = (i + offset_latitude) * global_world.longitude
                          + (j + offset_longitude);
        albedo[i * longitude + j] = global_world.albedo_data[k_global];
    }

    // create local world data
    World world = World(latitude, longitude, 293.15, albedo);
    world.global_latitude  = global_world.latitude;
    world.global_longitude = global_world.longitude;
    world.offset_latitude  = offset_latitude;
    world.offset_longitude = offset_longitude;

    // set up counters and loop for num_iterations of integration steps
    const double delta_time = world.global_longitude / 36.0;

    std::vector<double> temp(global_world.latitude * global_world.longitude);

    //Comment: We define a new datatype, which is nothing else than a column. This facilitates the exchange of the ghost lines.
    MPI_Type_vector(world.latitude, 1, world.longitude, MPI_DOUBLE, &column);
    MPI_Type_commit(&column);

    auto begin = std::chrono::steady_clock::now();

    for (uint64_t iteration=0; iteration < num_of_iterations; ++iteration) {
        world.time = iteration / delta_time;
        integrate(world);

        // TODO: gather the Temperature on rank zero
        // Comment: We set everything to zero, so that we can simply add all global_world.data to get the global data.
        for (uint64_t i = 0; i < world.global_latitude; ++i)
        for (uint64_t j = 0; j < world.global_longitude; ++j) {
            global_world.data[i*world.global_longitude + j] = 0.0;
        }

        // remove ghostzones and construct global data from local data
        for (uint64_t i = 1; i < latitude-1; ++i)
        for (uint64_t j = 1; j < longitude-1; ++j) {
            uint64_t k_global = (i + offset_latitude) * global_world.longitude
                              + (j + offset_longitude);
            global_world.data[k_global] = world.data[i * longitude + j];
        }

        // Comment: We 'gather' all data from the local global_world.data to the temp by just adding them.
        MPI_Reduce(&global_world.data[0], &temp[0], global_world.latitude * global_world.longitude, MPI_DOUBLE, MPI_SUM, 0, MPI_COMM_WORLD);

        if (!output_filename.empty()) {
```

```
            // Only rank zero writes water history to file
            if (mpi_rank == 0) {
                // Comment: We copy the temperature data to the global_world.data.
                global_world.data = temp;
                if (iteration%100==0){
                    write_hdf5(global_world, output_filename, iteration);
                }
                std::cout << iteration << " -- ";
                stat(global_world);
            }
        }
    }
    auto end = std::chrono::steady_clock::now();

    double local_cs, global_cs, local_time;
    stat(world);
    local_cs = checksum(world);
    local_time = (end - begin).count() / 1000000000.0;
    double global_time;

    // Comment: We compute the global checksum by adding all local checksums.
    MPI_Reduce(&local_cs, &global_cs, 1, MPI_DOUBLE, MPI_SUM, 0, MPI_COMM_WORLD);
    // Comment: We compute the elapsed time as the elapsed time of the slowest rank.
    MPI_Reduce(&local_time, &global_time, 1, MPI_DOUBLE, MPI_MAX, 0, MPI_COMM_WORLD);

    if(mpi_rank==0){
    std::cout << "global checksum      : " << global_cs << std::endl;
    std::cout << "Elapsed time of the slowest rank  : " << global_time << " sec" <<
    std::endl;
    }
}

/** Main function that parses the command line and start the simulation */
int main(int argc, char **argv) {

    // Initialize the MPI environment
    MPI_Init(&argc, &argv);

    // Get the number of processes
    MPI_Comm_size(MPI_COMM_WORLD, &mpi_size);

    // Get the rank of the process
    MPI_Comm_rank(MPI_COMM_WORLD, &mpi_rank);

    // Get the name of the processor
    char processor_name[MPI_MAX_PROCESSOR_NAME];
    int name_len;
    MPI_Get_processor_name(processor_name, &name_len);

    // Print off a hello world message
    std::cout << "Flat World Climate running on " << processor_name
              << ", rank " << mpi_rank << " out of " << mpi_size << std::endl;

    uint64_t iterations=0;
    std::string model_filename;
    std::string output_filename;

    std::vector <std::string> argument({argv, argv+argc});

    for (long unsigned int i = 1; i<argument.size() ; i += 2){
        std::string arg = argument[i];
        if(arg=="-h"){ // Write help
```

```
            std::cout << "./fwc --iter <number of iterations>"
                      << " --model <input model>"
                      << " --out <name of output file>\n";
            exit(0);
        } else if (i == argument.size() - 1)
            throw std::invalid_argument("The last argument (" + arg +") must have a value");
        else if(arg=="--iter"){
            if ((iterations = std::stoi(argument[i+1])) < 0)
                throw std::invalid_argument("iter most be positive (e.g. -iter 1000)");
        } else if(arg=="--model"){
            model_filename = argument[i+1];
        } else if(arg=="--out"){
            output_filename = argument[i+1];
        } else{
            std::cout << "---> error: the argument type is not recognized \n";
        }
    }
    if (model_filename.empty())
        throw std::invalid_argument("You must specify the model to simulate "
                                    "(e.g. --model models/small.hdf5)");
    if (iterations==0)
        throw std::invalid_argument("You must specify the number of iterations "
                                    "(e.g. --iter 10)");

    simulate(iterations, model_filename, output_filename);

    MPI_Finalize();

    return 0;
}
```

Complete code for the 1D–slab geometry.

```
#include <vector>
#include <iostream>
#include <H5Cpp.h>
#include <chrono>
#include <cmath>
#include <numeric>
#include <mpi.h>

// Get the number of processes
int mpi_size;

// Get the rank of the process
int mpi_rank;

MPI_Comm grid_comm;
/** Representation of a flat world */
class World {
public:
    // The current world time of the world
    double time;
    // The size of the world in the latitude dimension and the global size
    uint64_t latitude, global_latitude;
    // The size of the world in the longitude dimension
    uint64_t longitude, global_longitude;
    // The offset for this rank in the latitude dimension
    long int offset_latitude;
    // The offset for this rank in the longitude dimension
    long int offset_longitude;
```

```
    // The temperature of each coordinate of the world.
    // NB: it is up to the calculation to interpret this vector in two dimension.
    std::vector<double> data;
    // The measure of the diffuse reflection of solar radiation at each world coordinate.
    // See: <https://en.wikipedia.org/wiki/Albedo>
    // NB: this vector has the same length as ‘data‘ and must be interpreted in two dimension as well.
    std::vector<double> albedo_data;

    /** Create a new flat world.
     *
     * @param latitude     The size of the world in the latitude dimension.
     * @param longitude    The size of the world in the longitude dimension.
     * @param temperature  The initial temperature (the whole world starts with the same temperature).
     * @param albedo_data  The measure of the diffuse reflection of solar radiation at each world coordinate.
     *                     This vector must have the size: ‘latitude * longitude‘.
     */
    World(uint64_t latitude, uint64_t longitude, double temperature,
          std::vector<double> albedo_data) : latitude(latitude), longitude(longitude),
                                             data(latitude * longitude, temperature),
                                             albedo_data(std::move(albedo_data)) {}
};

double checksum(World &world) {
    //
    double cs=0;
    // TODO: make sure checksum is computed globally
    // only loop *inside* data region -- not in ghostzones!
    for (uint64_t i = 1; i < world.latitude - 1; ++i)
    for (uint64_t j = 1; j < world.longitude - 1; ++j) {
        cs += world.data[i*world.longitude + j];
    }
    return cs;
}

void stat(World &world) {
    // TODO: make sure stats are computed globally
    double mint = 1e99;
    double maxt = 0;
    double meant = 0;
    double global_mint = 0;
    double global_maxt = 0;
    double global_meant = 0;
    double size = world.global_latitude * world.global_longitude;

    if(world.data.size() < world.global_latitude * world.global_longitude){
    for (uint64_t i = 1; i < world.latitude - 1; ++i)
    for (uint64_t j = 1; j < world.longitude - 1; ++j) {
        mint = std::min(mint,world.data[i*world.longitude + j]);
        maxt = std::max(maxt,world.data[i*world.longitude + j]);
        meant += world.data[i*world.longitude + j];
    }
    meant = meant / size;

    MPI_Reduce(&meant, &global_meant, 1, MPI_DOUBLE, MPI_SUM, 0, grid_comm);
    MPI_Reduce(&mint, &global_mint, 1, MPI_DOUBLE, MPI_MIN, 0  , grid_comm);
    MPI_Reduce(&maxt, &global_maxt, 1, MPI_DOUBLE, MPI_MAX, 0  , grid_comm);

```

```
    }
    else{
        size = world.latitude*world.longitude;
        for (uint64_t i = 0; i < world.latitude; ++i)
        for (uint64_t j = 0; j < world.longitude; ++j) {
        mint = std::min(mint,world.data[i*world.longitude + j]);
        maxt = std::max(maxt,world.data[i*world.longitude + j]);
        meant += world.data[i*world.longitude + j];
        }
        // std::cout<<"mean is"<<meant<<"\n size is"<<size<<std::endl;
        meant = meant / size;

        global_mint = mint;
        global_maxt = maxt;
        global_meant = meant;
        }
    if(mpi_rank == 0){
    std::cout <<   "min: " << global_mint
              << ", max: " << global_maxt
              << ", avg: " << global_meant << std::endl;
    }
}

/** Exchange the ghost cells i.e. copy the second data row and column to the very last data row and column and vice versa.
 *
 * @param world  The world to fix the boundaries for.
 */
void exchange_ghost_cells(World &world) {

    MPI_Request req[8];
    int tag = 0;

    double emit_down[world.longitude],emit_up[world.longitude],emit_left[world.latitude],emit_right[world.latitude];
    double recv_left[world.latitude], recv_right[world.latitude], recv_up[world.longitude], recv_down[world.longitude];

    int neighb_down[2]; int neighb_up[2];
    int displacement=1;

    for(int direction=0; direction<2; direction++)
        MPI_Cart_shift(grid_comm, direction, displacement , &neighb_down[direction],&neighb_up[direction]);

    for(uint64_t i=0; i<world.longitude; ++i){
        emit_left[i]  = world.data[i * world.latitude + 1];
        emit_right[i] = world.data[i * world.latitude + world.longitude - 2];
    }

    for (uint64_t i = 1; i < world.longitude - 1; ++i) {
        emit_up[i] = world.data[i + world.longitude];
        emit_down[i] = world.data[i + world.longitude*(world.latitude-2)];
    }

    //left-right
    MPI_Isend(&emit_left[0], world.latitude, MPI_DOUBLE, neighb_up[1], tag, MPI_COMM_WORLD, &req[0]);
    MPI_Isend(&emit_right[0], world.latitude, MPI_DOUBLE, neighb_down[1], tag , MPI_COMM_WORLD, &req[1]);
    // up-down
    MPI_Isend(&emit_up[0], world.longitude, MPI_DOUBLE, neighb_down[0], tag,
```

```
    MPI_COMM_WORLD, &req[2]);
    MPI_Isend(&emit_down[0], world.longitude, MPI_DOUBLE, neighb_up[0], tag, MPI_COMM_WORLD, &req[3]);

    // Receive from left and right
    MPI_Irecv(&recv_left[0], world.latitude, MPI_DOUBLE, neighb_down[0], tag, MPI_COMM_WORLD, &req[4]);
    MPI_Irecv(&recv_right[0], world.latitude, MPI_DOUBLE, neighb_up[0], tag , MPI_COMM_WORLD, &req[5]);

    // Receive from up and down
    MPI_Irecv(&recv_up[0], world.longitude, MPI_DOUBLE, neighb_up[1], tag, MPI_COMM_WORLD, &req[6]);
    MPI_Irecv(&recv_down[0], world.longitude, MPI_DOUBLE, neighb_down[1], tag, MPI_COMM_WORLD, &req[7]);

    MPI_Waitall(8, &req[0], MPI_STATUSES_IGNORE);

    // Copy received data to appropriate locations in world.data
    for (uint64_t i = 0; i < world.longitude; ++i) {

        world.data[i * world.latitude] = recv_left[i];
        world.data[i * world.latitude + world.longitude - 1] = recv_right[i];

    }

    for (uint64_t i = 1; i < world.longitude - 1; ++i) {

        world.data[i]=recv_up[i];
        world.data[(i + world.longitude*(world.latitude-1))] = recv_down[i];

    }
}

/** Warm the world based on the position of the sun.
 *
 * @param world      The world to warm.
 */
void radiation(World& world) {
    double sun_angle = std::cos(world.time);
    double sun_intensity = 865.0;
    double sun_long = (std::sin(sun_angle) * (world.global_longitude / 2))
                      + world.global_longitude / 2.;
    double sun_lat = world.global_latitude / 2.;
    double sun_height = 100. + std::cos(sun_angle) * 100.;
    double sun_height_squared = sun_height * sun_height;

    for (uint64_t i = 1; i < world.latitude-1; ++i) {
        for (uint64_t j = 1; j < world.longitude-1; ++j) {
            // Euclidean distance between the sun and each earth coordinate
            double delta_lat  = sun_lat  - (i + world.offset_latitude);
            double delta_long = sun_long - (j + world.offset_longitude);
            double dist = sqrt(delta_lat*delta_lat +
                               delta_long*delta_long +
                               sun_height_squared);
            world.data[i * world.longitude + j] += \
                (sun_intensity / dist) * (1. - world.albedo_data[i * world.longitude + j]);
        }
    }
    exchange_ghost_cells(world);
}
```

```

/** Heat radiated to space
 *
 * @param world  The world to update.
 */
void energy_emmision(World& world) {
    for (uint64_t i = 0; i < world.latitude * world.longitude; ++i) {
        world.data[i] *= 0.99;
    }
}

/** Heat diffusion
 *
 * @param world  The world to update.
 */
void diffuse(World& world) {
    std::vector<double> tmp = world.data;
    for (uint64_t k = 0; k < 10; ++k) {
        for (uint64_t i = 1; i < world.latitude - 1; ++i) {
            for (uint64_t j = 1; j < world.longitude - 1; ++j) {
                // 5 point stencil
                double center = world.data[i * world.longitude + j];
                double left = world.data[(i - 1) * world.longitude + j];
                double right = world.data[(i + 1) * world.longitude + j];
                double up = world.data[i * world.longitude + (j - 1)];
                double down = world.data[i * world.longitude + (j + 1)];
                tmp[i * world.longitude + j] = (center + left + right + up + down) / 5.;
            }
        }
        std::swap(world.data, tmp);  // swap pointers for the two arrays
        exchange_ghost_cells(world); // update ghost zones
    }
}

/** One integration step at ‘world_time‘
 *
 * @param world      The world to update.
 */
void integrate(World& world) {
    radiation(world);
    energy_emmision(world);
    diffuse(world);
}

/** Read a world model from a HDF5 file
 *
 * @param filename The path to the HDF5 file.
 * @return         A new world based on the HDF5 file.
 */
World read_world_model(const std::string& filename) {
    H5::H5File file(filename, H5F_ACC_RDONLY);
    H5::DataSet dataset = file.openDataSet("world");
    H5::DataSpace dataspace = dataset.getSpace();

    if (dataspace.getSimpleExtentNdims() != 2) {
        throw std::invalid_argument("Error while reading the model: the number of dimension must be two.");
    }

    if (dataset.getTypeClass() != H5T_FLOAT or dataset.getFloatType().getSize() != 8) {
```

```
        throw std::invalid_argument("Error while reading the model: wrong data type, must be double.");
    }

    hsize_t dims[2];
    dataspace.getSimpleExtentDims(dims, NULL);
    std::vector<double> data_out(dims[0] * dims[1]);
    dataset.read(data_out.data(), H5::PredType::NATIVE_DOUBLE, dataspace, dataspace);
    std::cout << "World model loaded -- latitude: " << (unsigned long) (dims[0]) << ", longitude: "
              << (unsigned long) (dims[1]) << std::endl;
    return World(static_cast<uint64_t>(dims[0]), static_cast<uint64_t>(dims[1]), 293.15, std::move(data_out));
}

/** Write data to a hdf5 file
 *
 *  @param group  The hdf5 group to write in
 * @param name   The name of the data
 * @param shape  The shape of the data
 * @param data   The data
 */
void write_hdf5_data(H5::Group& group, const std::string& name,
                     const std::vector <hsize_t>& shape, const std::vector<double>& data) {
    H5::DataSpace dataspace(static_cast<int>(shape.size()), &shape[0]);
    H5::DataSet dataset = group.createDataSet(name.c_str(), H5::PredType::NATIVE_DOUBLE, dataspace);
    dataset.write(&data[0], H5::PredType::NATIVE_DOUBLE);
}

/** Write a history of the world temperatures to a HDF5 file
 *s
 * @param world     world to write
 * @param filename  The output filename of the HDF5 file
 */
void write_hdf5(const World& world, const std::string& filename, uint64_t iteration) {

    static H5::H5File file(filename, H5F_ACC_TRUNC);

    H5::Group group(file.createGroup("/" + std::to_string(iteration)));
    write_hdf5_data(group, "world", {world.latitude, world.longitude}, world.data);
}

/** Simulation of a flat word climate
 *
 * @param num_of_iterations  Number of time steps to simulate
 * @param model_filename     The filename of the world model to use (HDF5 file)
 * @param output_filename    The filename of the written world history (HDF5 file)
 */
void simulate(uint64_t num_of_iterations, const std::string& model_filename, const std::string& output_filename) {

    // for simplicity, read in full model
    World global_world = read_world_model(model_filename);
    // TODO: compute offsets according to rank and domain decomposition
    // figure out size of domain for this rank

    int dims[2]= {0,0};
    MPI_Dims_create(mpi_size, 2,dims);

```

```
    int periods[2] = {1,1};
    MPI_Cart_create(MPI_COMM_WORLD, 2, dims, periods, 1, &grid_comm);


    int remlon = global_world.longitude % dims[0];
    int divilon = global_world.longitude / dims[0];
    int remlat = global_world.latitude % dims[1];
    int divilat = global_world.latitude / dims[1];


    // Comment: We divide our grid in 'divi' parts. If there is a remainder, we distribute it equally amongst the ranks in order.
    const uint64_t longitude = (mpi_rank < remlon) ? divilon + 3 : divilon + 2; // one ghost cell on each end
    // Comment: We compute the offset of each rank by considering the remainder.
    const long int offset_longitude = -1 + mpi_rank * divilon + ((mpi_rank < remlon) ? mpi_rank : remlon); // -1 because first cell is a ghostcell;

    // Comment: Latitute and longitude offset do not change.

    const uint64_t latitude  = (mpi_rank < remlat) ? divilat + 3 : divilat + 2;
    const long int offset_latitude  = -1 + mpi_rank * divilat + ((mpi_rank < remlat) ? mpi_rank : remlat);

    // copy over albedo data to local world data
    std::vector<double> albedo(longitude*latitude);
    for (uint64_t i = 1; i < latitude-1; ++i)
    for (uint64_t j = 1; j < longitude-1; ++j) {
        uint64_t k_global = (i + offset_latitude) * global_world.longitude
                          + (j + offset_longitude);
        albedo[i * longitude + j] = global_world.albedo_data[k_global];
    }

    // create local world data
    World world = World(latitude, longitude, 293.15, albedo);
    world.global_latitude  = global_world.latitude;
    world.global_longitude = global_world.longitude;
    world.offset_latitude  = offset_latitude;
    world.offset_longitude = offset_longitude;

    // set up counters and loop for num_iterations of integration steps
    const double delta_time = world.global_longitude / 36.0;

    std::vector<double> temp(global_world.latitude * global_world.longitude);

    auto begin = std::chrono::steady_clock::now();

    for (uint64_t iteration=0; iteration < num_of_iterations; ++iteration) {
        world.time = iteration / delta_time;
        integrate(world);

        // TODO: gather the Temperature on rank zero
        // Comment: We set everything to zero, so that we can simply add all global_world.data to get the global data.
        for (uint64_t i = 0; i < world.global_latitude; ++i)
        for (uint64_t j = 0; j < world.global_longitude; ++j) {
            global_world.data[i*world.global_longitude + j] = 0.0;
        }

        // remove ghostzones and construct global data from local data
        for (uint64_t i = 1; i < latitude-1; ++i)
        for (uint64_t j = 1; j < longitude-1; ++j) {
```

```
            uint64_t k_global = (i + offset_latitude) * global_world.longitude
                                + (j + offset_longitude);
            global_world.data[k_global] = world.data[i * longitude + j];
        }

        // Comment: We 'gather' all data from the local global_world.data to the temp by just adding them.
        MPI_Reduce(&global_world.data[0], &temp[0], global_world.latitude * global_world.longitude, MPI_DOUBLE, MPI_SUM, 0, grid_comm);

        if (!output_filename.empty()) {
            // Only rank zero writes water history to file
            if (mpi_rank == 0) {
                // Comment: We copy the temperature data to the global_world.data.
                global_world.data = temp;
                if (iteration%100==0){
                    write_hdf5(global_world, output_filename, iteration);
                }
                std::cout << iteration << " -- ";
                stat(global_world);
            }
        }
    }
    auto end = std::chrono::steady_clock::now();

    double local_cs, global_cs, local_time;
    stat(world);
    local_cs = checksum(world);
    local_time = (end - begin).count() / 1000000000.0;
    double global_time;

    MPI_Reduce(&local_cs, &global_cs, 1, MPI_DOUBLE, MPI_SUM, 0, grid_comm);
    MPI_Reduce(&local_time, &global_time, 1, MPI_DOUBLE, MPI_MAX, 0,grid_comm);

    if(mpi_rank==0){
    std::cout << "global checksum      : " << global_cs << std::endl;
    std::cout << "Elapsed time of the slowest rank  : " << global_time << " sec" << std::endl;
    }
}

/** Main function that parses the command line and start the simulation */
int main(int argc, char **argv) {

    // Initialize the MPI environment
    MPI_Init(&argc, &argv);

    // Get the number of processes
    MPI_Comm_size(MPI_COMM_WORLD, &mpi_size);

    // Get the rank of the process
    MPI_Comm_rank(MPI_COMM_WORLD, &mpi_rank);

    // Get the name of the processor
    char processor_name[MPI_MAX_PROCESSOR_NAME];
    int name_len;
    MPI_Get_processor_name(processor_name, &name_len);

    // Print off a hello world message
    std::cout << "Flat World Climate running on " << processor_name
              << ", rank " << mpi_rank << " out of " << mpi_size << std::endl;

```

```
    uint64_t iterations=0;
    std::string model_filename;
    std::string output_filename;

    std::vector <std::string> argument({argv, argv+argc});

    for (long unsigned int i = 1; i<argument.size() ; i += 2){
        std::string arg = argument[i];
        if(arg=="-h"){ // Write help
            std::cout << "./fwc --iter <number of iterations>"
                      << " --model <input model>"
                      << " --out <name of output file>\n";
            exit(0);
        } else if (i == argument.size() - 1)
            throw std::invalid_argument("The last argument (" + arg +") must have a value");
        else if(arg=="--iter"){
            if ((iterations = std::stoi(argument[i+1])) < 0)
                throw std::invalid_argument("iter most be positive (e.g. -iter 1000)");
        } else if(arg=="--model"){
            model_filename = argument[i+1];
        } else if(arg=="--out"){
            output_filename = argument[i+1];
        } else{
            std::cout << "---> error: the argument type is not recognized \n";
        }
    }
    if (model_filename.empty())
        throw std::invalid_argument("You must specify the model to simulate "
                                    "(e.g. --model models/small.hdf5)");
    if (iterations==0)
        throw std::invalid_argument("You must specify the number of iterations "
                                    "(e.g. --iter 10)");

    simulate(iterations, model_filename, output_filename);

    MPI_Finalize();

    return 0;
}
```

Complete code for the 2D domain decomposition.

```
#include <vector>
#include <iostream>
#include <H5Cpp.h>
#include <chrono>
#include <cmath>
#include <numeric>
#include <mpi.h>
#include <omp.h>

// Get the number of processes
int mpi_size;

// Get the rank of the process
int mpi_rank;
MPI_Datatype column;
/** Representation of a flat world */
class World {
public:
    // The current world time of the world
```

```
    double time;
    // The size of the world in the latitude dimension and the global size
    uint64_t latitude, global_latitude;
    // The size of the world in the longitude dimension
    uint64_t longitude, global_longitude;
    // The offset for this rank in the latitude dimension
    long int offset_latitude;
    // The offset for this rank in the longitude dimension
    long int offset_longitude;
    // The temperature of each coordinate of the world.
    // NB: it is up to the calculation to interpret this vector in two dimension.
    std::vector<double> data;
    // The measure of the diffuse reflection of solar radiation at each world coordinate.
    // See: <https://en.wikipedia.org/wiki/Albedo>
    // NB: this vector has the same length as `data` and must be interpreted in two dimension as well.
    std::vector<double> albedo_data;

    /** Create a new flat world.
     *
     * @param latitude     The size of the world in the latitude dimension.
     * @param longitude    The size of the world in the longitude dimension.
     * @param temperature  The initial temperature (the whole world starts with the same temperature).
     * @param albedo_data  The measure of the diffuse reflection of solar radiation at each world coordinate.
     *                     This vector must have the size: `latitude * longitude`.
     */
    World(uint64_t latitude, uint64_t longitude, double temperature,
          std::vector<double> albedo_data) : latitude(latitude), longitude(longitude),
                                             data(latitude * longitude, temperature),
                                             albedo_data(std::move(albedo_data)) {}
};

double checksum(World &world) {
    //
    double cs=0;
    // TODO: make sure checksum is computed globally
    // only loop *inside* data region -- not in ghostzones!
    for (uint64_t i = 1; i < world.latitude - 1; ++i)
    for (uint64_t j = 1; j < world.longitude - 1; ++j) {
        cs += world.data[i*world.longitude + j];
    }
    return cs;
}

void stat(World &world) {
    // TODO: make sure stats are computed globally
    double mint = 1e99;
    double maxt = 0;
    double meant = 0;
    double global_mint = 0;
    double global_maxt = 0;
    double global_meant = 0;
    double size = world.global_latitude * world.global_longitude;

    if(world.data.size() < world.global_latitude * world.global_longitude){
    for (uint64_t i = 1; i < world.latitude - 1; ++i)
    for (uint64_t j = 1; j < world.longitude - 1; ++j) {
        mint = std::min(mint,world.data[i*world.longitude + j]);
```

```
        maxt = std::max(maxt,world.data[i*world.longitude + j]);
        meant += world.data[i*world.longitude + j];
    }
    meant = meant / size;

    MPI_Reduce(&meant, &global_meant, 1, MPI_DOUBLE, MPI_SUM, 0, MPI_COMM_WORLD);
    MPI_Reduce(&mint, &global_mint, 1, MPI_DOUBLE, MPI_MIN, 0, MPI_COMM_WORLD);
    MPI_Reduce(&maxt, &global_maxt, 1, MPI_DOUBLE, MPI_MAX, 0, MPI_COMM_WORLD);

    }
    else{
        size = world.latitude*world.longitude;
        for (uint64_t i = 0; i < world.latitude; ++i)
        for (uint64_t j = 0; j < world.longitude; ++j) {
        mint = std::min(mint,world.data[i*world.longitude + j]);
        maxt = std::max(maxt,world.data[i*world.longitude + j]);
        meant += world.data[i*world.longitude + j];
        }
        // std::cout<<"mean is"<<meant<<"\n size is"<<size<<std::endl;
        meant = meant / size;

        global_mint = mint;
        global_maxt = maxt;
        global_meant = meant;
        }
    if(mpi_rank == 0){
    std::cout <<   "min: " << global_mint
              << ", max: " << global_maxt
              << ", avg: " << global_meant << std::endl;
    }
}

/** Exchange the ghost cells i.e. copy the second data row and column to the very last data row and column and vice versa.
 *
 * @param world  The world to fix the boundaries for.
 */
void exchange_ghost_cells(World &world) {
    // TODO: figure out exchange of ghost cells bewteen ranks
    // Comment: Exchange of the ghost lines by sending and receiving data.

    MPI_Request req[4];
    int tag = 0;

    int left = (mpi_rank - 1 + mpi_size) % mpi_size;
    int right = (mpi_rank +  1) % mpi_size;

    MPI_Isend(&world.data[1], 1, column, left, tag, MPI_COMM_WORLD, &req[0]);
    MPI_Isend(&world.data[world.longitude - 2], 1, column, right, tag, MPI_COMM_WORLD, &req[1]);
    MPI_Irecv(&world.data[0], 1, column, left, tag, MPI_COMM_WORLD, &req[2]);
    MPI_Irecv(&world.data[world.longitude - 1], 1, column, right, tag, MPI_COMM_WORLD, &req[3]);
    MPI_Waitall(4, &req[0], MPI_STATUSES_IGNORE);

    for (uint64_t j = 0; j < world.longitude; ++j) {
        world.data[0*world.longitude + j] = world.data[(world.latitude-2)*world.longitude + j];
        world.data[(world.latitude-1)*world.longitude + j] = world.data[1*world.longitude + j];
    }

```

```
}

/** Warm the world based on the position of the sun.
 *
 * @param world      The world to warm.
 */
void radiation(World& world) {
    double sun_angle = std::cos(world.time);
    double sun_intensity = 865.0;
    double sun_long = (std::sin(sun_angle) * (world.global_longitude / 2))
                      + world.global_longitude / 2.;
    double sun_lat = world.global_latitude / 2.;
    double sun_height = 100. + std::cos(sun_angle) * 100.;
    double sun_height_squared = sun_height * sun_height;
    #pragma omp parallel for collapse(2)
    for (uint64_t i = 1; i < world.latitude-1; ++i) {
        for (uint64_t j = 1; j < world.longitude-1; ++j) {
            // Euclidean distance between the sun and each earth coordinate
            double delta_lat  = sun_lat  - (i + world.offset_latitude);
            double delta_long = sun_long - (j + world.offset_longitude);
            double dist = sqrt(delta_lat*delta_lat +
                               delta_long*delta_long +
                               sun_height_squared);
            world.data[i * world.longitude + j] += \
                (sun_intensity / dist) * (1. - world.albedo_data[i * world.longitude + j]);
        }
    }
    exchange_ghost_cells(world);
}

/** Heat radiated to space
 *
 * @param world  The world to update.
 */
void energy_emmision(World& world) {
    #pragma omp parallel for
    for (uint64_t i = 0; i < world.latitude * world.longitude; ++i) {
        world.data[i] *= 0.99;
    }
}

/** Heat diffusion
 *
 * @param world  The world to update.
 */
void diffuse(World& world) {
    std::vector<double> tmp = world.data;
    for (uint64_t k = 0; k < 10; ++k) {
        #pragma omp parallel for collapse(2)
        for (uint64_t i = 1; i < world.latitude - 1; ++i) {
            for (uint64_t j = 1; j < world.longitude - 1; ++j) {
                // 5 point stencil
                double center = world.data[i * world.longitude + j];
                double left = world.data[(i - 1) * world.longitude + j];
                double right = world.data[(i + 1) * world.longitude + j];
                double up = world.data[i * world.longitude + (j - 1)];
                double down = world.data[i * world.longitude + (j + 1)];
                tmp[i * world.longitude + j] = (center + left + right + up + down) / 5.;
            }
        }
```

```
        std::swap(world.data, tmp);  // swap pointers for the two arrays
        exchange_ghost_cells(world); // update ghost zones
    }
}

/** One integration step at `world_time`
 *
 * @param world      The world to update.
 */
void integrate(World& world){
    radiation(world);
    energy_emmision(world);
    diffuse(world);
}

/** Read a world model from a HDF5 file
 *
 * @param filename The path to the HDF5 file.
 * @return         A new world based on the HDF5 file.
 */
World read_world_model(const std::string& filename) {
    H5::H5File file(filename, H5F_ACC_RDONLY);
    H5::DataSet dataset = file.openDataSet("world");
    H5::DataSpace dataspace = dataset.getSpace();

    if (dataspace.getSimpleExtentNdims() != 2) {
        throw std::invalid_argument("Error while reading the model: the number of dimension must be two.");
    }

    if (dataset.getTypeClass() != H5T_FLOAT or dataset.getFloatType().getSize() != 8) {
        throw std::invalid_argument("Error while reading the model: wrong data type, must be double.");
    }

    hsize_t dims[2];
    dataspace.getSimpleExtentDims(dims, NULL);
    std::vector<double> data_out(dims[0] * dims[1]);
    dataset.read(data_out.data(), H5::PredType::NATIVE_DOUBLE, dataspace, dataspace);
    std::cout << "World model loaded -- latitude: " << (unsigned long) (dims[0]) << ", longitude: "
              << (unsigned long) (dims[1]) << std::endl;
    return World(static_cast<uint64_t>(dims[0]), static_cast<uint64_t>(dims[1]), 293.15, std::move(data_out));
}

/** Write data to a hdf5 file
 *
 *  @param group  The hdf5 group to write in
 * @param name   The name of the data
 * @param shape  The shape of the data
 * @param data   The data
 */
void write_hdf5_data(H5::Group& group, const std::string& name,
                     const std::vector <hsize_t>& shape, const std::vector<double>& data) {
    H5::DataSpace dataspace(static_cast<int>(shape.size()), &shape[0]);
    H5::DataSet dataset = group.createDataSet(name.c_str(), H5::PredType::NATIVE_DOUBLE, dataspace);
    dataset.write(&data[0], H5::PredType::NATIVE_DOUBLE);
}
```

```

/** Write a history of the world temperatures to a HDF5 file
 *s
 * @param world     world to write
 * @param filename  The output filename of the HDF5 file
 */
void write_hdf5(const World& world, const std::string& filename, uint64_t iteration) {

    static H5::H5File file(filename, H5F_ACC_TRUNC);

    H5::Group group(file.createGroup("/" + std::to_string(iteration)));
    write_hdf5_data(group, "world", {world.latitude, world.longitude}, world.data);
}

/** Simulation of a flat word climate
 *
 * @param num_of_iterations  Number of time steps to simulate
 * @param model_filename     The filename of the world model to use (HDF5 file)
 * @param output_filename    The filename of the written world history (HDF5 file)
 */
void simulate(uint64_t num_of_iterations, const std::string& model_filename, const std::string& output_filename) {

    // for simplicity, read in full model
    World global_world = read_world_model(model_filename);
    /*
    // TODO: compute offsets according to rank and domain decomposition
    // figure out size of domain for this rank
    */

    int rem, divi;

    rem = global_world.longitude % mpi_size;
    divi = global_world.longitude / mpi_size;
    // Comment: We divide our grid in 'divi' parts. If there is a remainder, we distribute it equally amongst the ranks in order.
    const uint64_t longitude = (mpi_rank < rem) ? divi + 3 : divi + 2; // one ghost cell on each end
     // Comment: We compute the offset of each rank by considering the remainder.
    const long int offset_longitude = -1 + mpi_rank * divi + ((mpi_rank < rem) ? mpi_rank : rem); // -1 because first cell is a ghostcell;

     // Comment: Latitute and longitude offset do not change.
    const uint64_t latitude  = global_world.latitude + 2;
    const long int offset_latitude  = -1;


    // copy over albedo data to local world data
    std::vector<double> albedo(longitude*latitude);
    for (uint64_t i = 1; i < latitude-1; ++i)
    for (uint64_t j = 1; j < longitude-1; ++j) {
        uint64_t k_global = (i + offset_latitude) * global_world.longitude
                            + (j + offset_longitude);
        albedo[i * longitude + j] = global_world.albedo_data[k_global];
    }

    // create local world data
    World world = World(latitude, longitude, 293.15, albedo);
    world.global_latitude  = global_world.latitude;
    world.global_longitude = global_world.longitude;
    world.offset_latitude  = offset_latitude;
```

```
    world.offset_longitude = offset_longitude;

    // set up counters and loop for num_iterations of integration steps
    const double delta_time = world.global_longitude / 36.0;

    std::vector<double> temp(global_world.latitude * global_world.longitude);

    //Comment: We define a new datatype, which is nothing else than a column. This facilitates the exchange of the ghost lines.
    MPI_Type_vector(world.latitude, 1, world.longitude, MPI_DOUBLE, &column);
    MPI_Type_commit(&column);

    auto begin = std::chrono::steady_clock::now();

    for (uint64_t iteration=0; iteration < num_of_iterations; ++iteration) {
        world.time = iteration / delta_time;
        integrate(world);

        // TODO: gather the Temperature on rank zero
        // Comment: We set everything to zero, so that we can simply add all global_world.data to get the global data.
        for (uint64_t i = 0; i < world.global_latitude; ++i)
        for (uint64_t j = 0; j < world.global_longitude; ++j) {
            global_world.data[i*world.global_longitude + j] = 0.0;
        }

        // remove ghostzones and construct global data from local data
        for (uint64_t i = 1; i < latitude-1; ++i)
        for (uint64_t j = 1; j < longitude-1; ++j) {
            uint64_t k_global = (i + offset_latitude) * global_world.longitude
                                + (j + offset_longitude);
            global_world.data[k_global] = world.data[i * longitude + j];
        }

        // Comment: We 'gather' all data from the local global_world.data to the temp by just adding them.
        MPI_Reduce(&global_world.data[0], &temp[0], global_world.latitude * global_world.longitude, MPI_DOUBLE, MPI_SUM, 0, MPI_COMM_WORLD);

        if (!output_filename.empty()) {
            // Only rank zero writes water history to file
            if (mpi_rank == 0) {
                // Comment: We copy the temperature data to the global_world.data.
                global_world.data = temp;
                if (iteration%100==0){
                    write_hdf5(global_world, output_filename, iteration);
                }
                std::cout << iteration << " -- ";
                stat(global_world);
            }
        }
    }
    auto end = std::chrono::steady_clock::now();

    double local_cs, global_cs, local_time;
    stat(world);
    local_cs = checksum(world);
    local_time = (end - begin).count() / 1000000000.0;
    double global_time;

    // Comment: We compute the global checksum by adding all local checksums.
    MPI_Reduce(&local_cs, &global_cs, 1, MPI_DOUBLE, MPI_SUM, 0, MPI_COMM_WORLD);
```

```
    // Comment: We compute the elapsed time as the elapsed time of the slowest rank.
    MPI_Reduce(&local_time, &global_time, 1, MPI_DOUBLE, MPI_MAX, 0, MPI_COMM_WORLD);

    if(mpi_rank==0){
    std::cout << "global checksum      : " << global_cs << std::endl;
    std::cout << "Elapsed time of the slowest rank  : " << global_time << " sec" << std::endl;
    }
}

/** Main function that parses the command line and start the simulation */
int main(int argc, char **argv) {

    // Initialize the MPI environment
    MPI_Init(&argc, &argv);

    // Get the number of processes
    MPI_Comm_size(MPI_COMM_WORLD, &mpi_size);

    // Get the rank of the process
    MPI_Comm_rank(MPI_COMM_WORLD, &mpi_rank);

    // Get the name of the processor
    char processor_name[MPI_MAX_PROCESSOR_NAME];
    int name_len;
    MPI_Get_processor_name(processor_name, &name_len);

    // Print off a hello world message
    std::cout << "Flat World Climate running on " << processor_name
              << ", rank " << mpi_rank << " out of " << mpi_size << std::endl;

    uint64_t iterations=0;
    std::string model_filename;
    std::string output_filename;

    std::vector <std::string> argument({argv, argv+argc});

    for (long unsigned int i = 1; i<argument.size() ; i += 2){
        std::string arg = argument[i];
        if(arg=="-h"){ // Write help
            std::cout << "./fwc --iter <number of iterations>"
                      << " --model <input model>"
                      << " --out <name of output file>\n";
            exit(0);
        } else if (i == argument.size() - 1)
            throw std::invalid_argument("The last argument (" + arg +") must have a value");
        else if(arg=="--iter"){
            if ((iterations = std::stoi(argument[i+1])) < 0)
                throw std::invalid_argument("iter most be positive (e.g. -iter 1000)");
        } else if(arg=="--model"){
            model_filename = argument[i+1];
        } else if(arg=="--out"){
            output_filename = argument[i+1];
        } else{
            std::cout << "---> error: the argument type is not recognized \n";
        }
    }
    if (model_filename.empty())
        throw std::invalid_argument("You must specify the model to simulate "
                                    "(e.g. --model models/small.hdf5)");
```

```
    if (iterations==0)
        throw std::invalid_argument("You must specify the number of iterations "
                                    "(e.g. --iter 10)");

    simulate(iterations, model_filename, output_filename);

    MPI_Finalize();

    return 0;
}
```

Complete code for hybrid 1D case.